

Análisis Numérico

Lógica computacional y sintaxis de Python

Ing. Eduardo Flores Rivas

Facultad de Ingeniería
Universidad Nacional Autónoma de México

Semestre 2026-2



Contenido

- 1 Pseudocódigo
- 2 Diagrama de flujo
- 3 Estructuras de programación
- 4 Python
- 5 Contacto



Pseudocódigo: El plan

¿Qué es pseudocódigo?

Descripción de alto nivel de un algoritmo usando una mezcla de lenguaje natural y convenciones de programación.

Características:

- No requiere sintaxis exacta
- Enfocado en la lógica
- Fácil de entender
- Paso intermedio entre problema y código

Convenciones comunes:

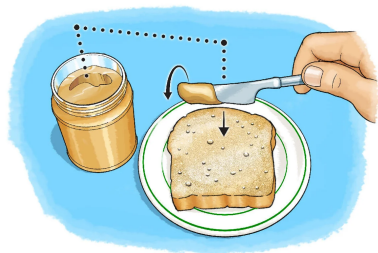
- INICIO/FIN o llaves {}
- LEER / ESCRIBIR
- SI...ENTONCES...SINO
- PARA, MIENTRAS, HACER
- # Comentarios

Ejemplo

¿Cómo preparar un Sandwich?



Video: Hacer un Sándwich



<https://www.youtube.com/watch?v=FN2RM-CHkuI>

Ejercicio mental

Mientras ves el video, identifica:

- **Entradas:** ¿Qué ingredientes necesita?
- **Procesos:** ¿Qué pasos sigue?
- **Salidas:** ¿Qué obtiene al final?
- **Decisiones:** ¿Toma alguna decisión durante el proceso?



Ejercicio: Preparar un sándwich

Escribir el pseudocódigo para el cálculo de la media aritmética de un conjunto de números.



Diagramas de flujo: Ver para entender

Símbolos estándar:

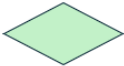
- **Óvalo:** Inicio/Fin
- **Rectángulo:** Proceso
- **Flechas:** Flujo
- **Rombo:** Decisión
- **Paralelogramo:** Entrada/Salida

Ventajas:

- Claridad visual
- Identificación de errores lógicos
- Comunicación efectiva
- Documentación

Symbol








Ejercicio: Diagrama de flujo

Para el pseudocódigo escrito en el ejercicio anterior, escribir el diagrama de flujo para el cálculo de la media aritmética.



Estructuras condicionales

¿Qué son las estructuras condicionales?

Permiten que un programa ejecute diferentes bloques de código según si una condición es verdadera o falsa.

Conceptos:

- **Condición:** Expresión que evalúa a True o False
- **Bloque de código:** Se ejecutan si la condición se cumple
- **Anidamiento:** Condiciones dentro de condiciones

Operadores de comparación:

- == Igual a
- != Diferente de
- < Menor que
- > Mayor que
- <= Menor o igual que
- >= Mayor o igual que

Operadores lógicos:

- and (Y lógico): Ambas verdaderas
- or (O lógico): Al menos una verdadera
- not (NO lógico): Negación



IF Simple y IF-ELSE

IF simple (solo si)

SI condición ENTONCES
 ejecutar bloque
FIN SI

IF-ELSE (si-sino)

SI condición ENTONCES
 bloque verdadero
SINO
 bloque falso
FIN SI



IF-ELIF-ELSE y CASE/SWITCH

IF-ELIF-ELSE (varias opciones)

```
SI condición1 ENTONCES
    bloque1
SINO SI condición2 ENTONCES
    bloque2
SINO SI condición3 ENTONCES
    bloque3
SINO
    bloque por defecto
FIN SI
```

SEGÚN/CASE (por valor)

```
SEGÚN valor HACER
    CASO 1:
        instrucción_1
        instrucción_2
    CASO 2:
        instrucción_3
        instrucción_4
    ...
    CASO SINO:
        instrucción_11
FIN SEGÚN
```

¿Cuándo usar cada uno?

- **IF-ELIF:** Para rangos, condiciones complejas, comparaciones
- **SEGÚN/CASE:** Para valores discretos exactos, enumeraciones, opciones de menú

¿Qué son las estructuras cíclicas?

Permiten ejecutar un bloque de código múltiples veces mientras se cumpla una condición o para cada elemento de una secuencia.

Tipos de bucles:

- **FOR:** Número de iteraciones conocido
- **WHILE:** Iterar hasta condición
- **DO-WHILE:** Ejecutar al menos una vez

Componentes esenciales:

- **Inicialización:** Valores iniciales
- **Condición:** Cuándo continuar/parar
- **Cuerpo:** Instrucciones a repetir
- **Actualización:** Cambio para avanzar



Bucle FOR

FOR (para)

PARA *variable* $\leftarrow$ *valor inicial* HASTA *valor final* CON PASO *incremento*
HACER
 instrucciones
FIN PARA

Ventajas del FOR

- Evita bucles infinitos
- Control explícito del número de iteraciones
- Fácil paralelización (potencial)
- Claridad en intención

Desventajas

Puede hacer iteraciones innecesarias si converge antes.



Bucle WHILE

WHILE (mientras)

MIENTRAS *condición* HACER
 instrucciones
FIN MIENTRAS

Ventajas del WHILE

- **Flexible:** No requiere conocer número de iteraciones
- **Natural:** Expresa condición lógica directamente
- **Potente:** Maneja casos complejos de terminación
- **Eficiente:** Solo ejecuta mientras sea necesario

Desventajas

- Riesgo de bucles infinitos
- Requiere modificar condición dentro del bucle
- Puede no ejecutarse ni una vez
- Más difícil de depurar



Bucle DO-WHILE

DO-WHILE (hacer-mientras)

HACER

instrucciones

MIENTRAS *condición*

Ventajas del DO-WHILE

- Garantiza al menos una ejecución
- Útil para validación de entrada
- Natural para procesos que deben iniciarse
- Evita duplicación de código inicial

Desventajas

- Menos común que WHILE
- No disponible en todos los lenguajes
- Condición evaluada al final (contraintuitivo)
- Mismo riesgo de bucles infinitos

Python: Creador y filosofía

Python es un lenguaje de programación creado por el neerlandés Guido van Rossum y publicado por primera vez en 1991.

The Zen of Python:

- *Bello es mejor que feo*
- *Explícito es mejor que implícito*
- *Simple es mejor que complejo*
- *La legibilidad cuenta*

Filosofía del diseño

Entonces, ¿qué van a hacer todos ustedes?

¿Crear una democracia? ¿Anarquía? ¿Una dictadura? ¿Una federación?

- **Guido van Rossum, 2018**

Benevolente Dictador Vitalicio



Python: ¿Por qué aprenderlo?

Para 2025 se posicionó como el lenguaje más popular según el índice TIOBE y el segundo más usado en GitHub.

¿Qué empresas lo usan?

- **OpenAI:** ChatGPT, GPT models
- **Google:** YouTube, Google Search
- **Netflix:** Recomendaciones, análisis
- **NASA:** Análisis de datos espaciales
- **Instagram:** Backend completo
- **Spotify:** Sistema de recomendación

Jan 2026	Jan 2025	Change	Programming Language	Ratings	Change
1	1		 Python	22.61%	-0.68%
2	4	▲	 C	10.99%	+2.13%
3	3		 Java	8.71%	-1.44%
4	2	▼	 C++	8.67%	-1.62%
5	5		 C#	7.39%	+2.94%
6	6		 JavaScript	3.03%	-1.17%
7	9	▲	 Visual Basic	2.41%	+0.04%
8	8		 SQL	2.27%	-0.14%
9	11	▲	 Delphi/Object Pascal	1.98%	+0.19%
10	18	▲	 R	1.82%	+0.81%

En ingeniería

Ampliamente usado en: simulación numérica, procesamiento de señales, control de sistemas y análisis de datos.

¿Por qué se volvió tan popular?

Python es un lenguaje con las siguientes características:

- **Código abierto:** Comunidad activa, no depende de una empresa
- **Interpretado:** Ejecución inmediata, ideal para prototipado
- **Alto nivel:** Sintaxis cercana al lenguaje natural
- **Propósito general:** Web, sistemas, ciencia de datos, IA, etc.
- **Baterías incluidas:** Biblioteca estándar extensa
- **Multiparadigma:** POO, funcional, imperativo
- **Multiplataforma:** Linux, Windows, macOS, embebidos
- **PIP + PyPI:** Gestor de paquetes + repositorio masivo
- **Fácil aprendizaje:** Curva suave para principiantes
- **Productividad:** Código conciso, rápido desarrollo



¿Qué es un tipo de dato?

Especifica qué valores puede tomar una variable y qué operaciones se pueden realizar con ella.

Tipos numéricos:

- `int` - Enteros
 - `x = 42`
 - `y = -15`
 - Precisión arbitraria
- `float` - Punto flotante
 - `x = 3.141592653589793`
 - `z = 1.5e-10` (científica)
 - Doble precisión (64 bits)
- `complex` - Complejos
 - `z = 3 + 4j`
 - `z = complex(2, -3)`
 - Esencial para raíces complejas

Otros tipos importantes:

- `bool` - Booleanos
 - `verdadero = True`
 - `falso = False`
 - Resultados de comparaciones
- `str` - Cadenas de texto
 - `nombre = "Newton-Raphson"`
 - Para mensajes y comandos
- `NoneType` - Valor nulo
 - `resultado = None`
 - Indica sin valor o no definido
 - Útil para inicialización



Contenedores para organizar datos

Estructuras que permiten almacenar múltiples valores.

list - Listas (mutables)

- `coeficientes = [1, -2, 5, -3]`
- `raices = [2.5, -1.3, 0.0]`
- Acceso: `lista[0]`, `lista[-1]`
- Modificación: `lista.append(4.2)`

tuple - Tuplas (inmutables)

- `punto = (3.5, 2.1)`
- `resultado = (1.618, True, 15)`
- Más eficientes que listas
- Útiles para retornar múltiples valores

dict - Diccionarios (clave-valor)

- `resultados = {"raiz": 2.5, "error": 1e-6, "iter": 15}`
- Acceso: `diccionario["clave"]`
- Ideal para resultados estructurados

set - Conjuntos (únicos, no ordenados)

- `unicos = {1, 2, 3, 3, 2} → {1, 2, 3}`
- Operaciones de conjuntos: unión, intersección
- Útil para eliminar duplicados



Diccionarios para resultados numéricos

Estructura ideal para resultados

Los diccionarios organizan datos en pares clave-valor.

Sintaxis básica

```
{clave1: valor1, clave2: valor2, ...}
```

Ejemplo completo:

```
resultado_newton = {  
    "metodo": "Newton-Raphson",  
    "raiz": 2.09455148,  
    "error": 1.5e-10,  
    "iteraciones": 5,  
    "convergencia": True,  
    "historial": [2.0, 2.1, 2.094, 2.09455]  
}
```

Tipos de valores permitidos:

- str (cadenas)
- float (números)
- int (enteros)
- bool (booleanos)
- list (listas)
- dict (otros diccionarios)



Manejo explícito de tipos

En métodos numéricos es crucial conocer y controlar los tipos de datos.

Ejemplo: Conversiones numéricas

```
# Conversiones comunes
x_int = 5
x_float = float(x_int)      # 5.0
x_complex = complex(x_float) # (5+0j)

# De cadena a número
str_val = "3.14159"
float_val = float(str_val)  # 3.14159

# Array NumPy desde lista
lista = [1, 2, 3, 4]
arr = np.array(lista, dtype=np.float64)
```

Conversión de tipos

- `float(x)` - A punto flotante
- `int(x)` - A entero (trunca)
- `complex(x)` - A complejo
- `str(x)` - A cadena
- `list(x)` - A lista
- `np.array(x)` - A array NumPy



Validar entrada

```
def metodo_newton(f, x0, tol=1e-8):  
    # Validar tipos  
    if not callable(f):  
        raise TypeError("f debe ser función")  
    if not isinstance(x0, (int, float)):  
        raise TypeError("x0 debe ser numérico")  
    if not isinstance(tol, float):  
        tol = float(tol) # Conversión automática  
    # Continuar con el método...
```

Precisión en métodos numéricos

- Usar `float()` para asegurar números de punto flotante
- Evitar comparaciones exactas: `x == 0.0`
- Usar tolerancias: `abs(x) < 1e-12`

Bucle FOR en Python

Sintaxis básica

```
for variable in secuencia:  
    # Bloque de instrucciones  
    # con sangría
```

Ejemplo con range()

```
# De 0 a 9  
for i in range(10):  
    print(f"Iteración {i}")  
  
# Con inicio, fin y paso  
for i in range(2, 20, 3):  
    print(i) # 2, 5, 8, 11, 14, 17
```

Consideraciones

- No tiene paso explícito en sintaxis
- Usa range() para control fino
- break para salida anticipada
- continue para saltar iteración

Ejemplo con lista

```
# Iterar sobre lista  
coeficientes = [1, -2, 3, -4]  
for coef in coeficientes:  
    print(f"Coeficiente: {coef}")  
  
# Con índice  
for i, coef in enumerate(coeficientes):  
    print(f"a[{i}] = {coef}")
```

Bucle WHILE en Python

Sintaxis básica

```
while condición:  
    # Bloque de instrucciones  
    # Actualizar condición
```

Ejemplo básico

```
# Contador con while  
contador = 1  
while contador <= 5:  
    print(f"Contador: {contador}")  
    contador += 1 # Actualización
```

Ejemplo con condiciones compuestas

```
# Múltiples condiciones  
error = 1.0  
iteraciones = 0  
max_iter = 1000  
while error > 1e-8 and iteraciones < max_iter:  
    # Cálculos aquí...  
    error = calcular_nuevo_error()  
    iteraciones += 1
```


Simulación DO-WHILE en Python

Patrón común (no hay sintaxis nativa)

```
while True:  # Bucle infinito aparente
    # Bloque de instrucciones
    # (se ejecuta al menos una vez)
    if not condición_continuar:
        break # Salir del bucle
```

Ejemplo: Validación de entrada

```
while True:
    numero = float(input("Ingrese positivo: "))

    if numero > 0:
        break # Entrada válida, salir

    print(";Inválido! Debe ser positivo.")
# Fuera del bucle: numero es positivo
```

Eduardo Flores Rivas
Ingeniero Mecatrónico
Facultad de Ingeniería, UNAM
eduardo.flores@ingenieria.unam.edu

